Assignment 2

The **goal** of this assignment is to implement a custom Gym environment, implement/apply Policy Iteration approach on the custom-built environment, and understand the limitation of classical dynamic programming approach.

Summary

1. Install the libraries needed
   - Python, Gymnasium, PyGame, NumPy
2. Create a custom Gym Environment for a “Grid Maze”
3. Create Environment Rendering using PyGame
4. Implement the “Policy Iteration” Dynamic Programming approach
5. Apply the Policy Iteration approach to the Grid Maze environment
6. Use “RecordVideo” Wrapper to record the RL agent in-action
7. Answer the following questions

Questions

1. What is the state-space size of the 5x5 Grid Maze problem?
2. How to optimize the policy iteration for the Grid Maze problem?
3. How many iterations did it take to converge on a stable policy for 5x5 maze?
4. Explain, with an example, how policy iteration behaves with multiple goal cells.
5. Can policy iteration work on a 100x100 maze? Explain why?
6. Can policy iteration work on a continuous-space maze? Explain why?
7. Can policy iteration work with moving bad cells (like Packman moving ghosts)? Explain why?

Deliverables

1. GitHub repository with Python codes (Gym Environment, Policy Iteration model)
2. Recorded videos of the trained agent in action
3. Report with the outcome summary and answers to the questions asked

Due date

26 Oct 2026

Helping Materials

- https://gymnasium.farama.org/index.html
- https://gymnasium.farama.org/introduction/create_custom_env/
- https://gymnasium.farama.org/tutorials/gymnasium_basics/environment_creation/
- https://gymnasium.farama.org/introduction/record_agent/
- https://gibberblot.github.io/rl-notes/single-agent/policy-iteration.html

Grid Maze Environment Description

The objective of this problem is to traverse a 5x5 grid-based maze from a random starting cell “S” to reach a randomly generated goal cell “G” while avoiding 2 randomly generated bad cells “X”. Fig 1 shows an example of the randomly generated maze, where the 4 cells are placed randomly.

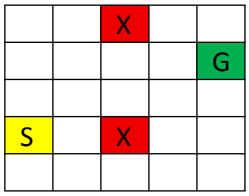


Fig 1: Example of a randomly generated maze

Observation Space

A set of 8 discrete integers representing the coordinates (x, y) of the current agent’s cell location, the goal cell location, and the 2 bad cell locations.

Action Space

A set of 4 discrete actions {right, up, left, down} indicating the movement direction. The agent moves in a deterministic way. Hence, taking the action “up” will move the agent one cell up.

Reward Function

The agent receives a reward according to the following:

- +1 if the agent reaches the goal cell
- -1 if the agent reaches the bad cell
- 0 otherwise